

Carl Brandenburg
Keller Brandenburg
CPSC 8470
2/5/2025
Project Phase 3: Final Report / Improve the Method
Repository for our modified source code
<https://github.com/Brande3/CPSC8470-Phase3>
Repository for the original source code
<https://github.com/WangXFng/CaDRec/tree/main>

“CaDRec: Contextualized and Debiased Recommender Model ”

In this final report we will discuss our experience with understanding, implementing, modifying, and attempting to improve the model as presented in the SIGIR '24 paper “CaDRec: Contextualized and Debiased Recommender Model.

Through phase 1 of our project we analyzed their paper to understand their goals and methods. The issues that the CaDRec team addressed were the over-smoothing of node embeddings which leads to a flattening of recommendations across users and the skewed distribution of interactions due to popularity and user-individual bias. They address the issue of over-smoothing of node embeddings by implementing a novel hypergraph convolution operator which is able to select effective neighbors during convolution by including both a structural and sequential context. They address the issue of skewed distributions through the modeling of individual biases to find unbiased item embeddings as well as using item popularity through the use of positional encoding. We found through reading their paper that in graph neural network recommender systems, embeddings tend to become similar as they are passed between nodes leading to recommendation being generalized rather than personalized. CaDRec improves in this area by selecting valuable neighbors during information transfer while adding additional meaning to the embeddings through the use of structural and sequential context. Additionally, self-attention perturbations are used to apply learned noise or randomness to adjust which neighbors are being used which assists in personalizing the system's recommendations. CaDRec is better able to handle bias by using positional encoding to monitor how popular an item is so that one very popular item is not able to dominate the recommendations. Additionally, it learns individual user ratings to learn the quality of an item from reviewer scores. This allows CaDRec to be fair in its presentation of items for recommendation. On the other hand, CaDRec relies heavily on the tuning of its hyperparameters which will be different for each dataset. Improper tuning can lead to problems such as over-regularization that leads to under-fitting, under-regularization that leads to large popularity bias, or imbalanced user and popularity bias. Other issues include but are not limited to, assuming that sequential context is always useful where if it is not this feature leads to needless complexity in the model and assuming that user preferences do not change over time which over a long enough period of time will lead to reduced performance and instead reliance on popularity of items over user preference.

To end phase 1, we suggested potential solutions to some of CaDRec's weaknesses. These include examining CaDRec's weakness to changing user preferences which we would attempt to address through implementing time-weighted embeddings in the model that would

reduce how much an item is affecting a user’s recommendation based on the amount of time since the user last interacted with it. We also suggested addressing the scalability of the model to make it more efficient through the replacing of the hypergraph convolutions with sparse attention convolutions alongside a method to prune weak or unimportant information.

During phase 2, we implemented the original CaDRec model’s source code as provided through their github repository. The implementation of the model was facilitated through the use of the Palmetto On Demand cluster to gain access to high end GPUs. On the cluster’s computers, the modules anaconda3/2023.09-0, cuda/12.3.0, and ngc/pytorch/23.06 were just about everything needed to successfully run the base model. The only additional libraries that were needed were optuna and sqlalchemy, the install instructions for use on the cluster can be found in our readme. After successfully running CaDRec, we began running the experiments as they were done in the paper so that we could compare our results to theirs. In the paper, recall (R) and normalized discounted cumulative gain (NDCG) were the primary performance metrics. In this context, R represents how many of the relevant items are in the top K results and NDCG represents how well the top K items were ranked. The goal with the metrics is to have as many relevant items as possible ranked in the proper order in the top K items.

Datasets	K	5	10	20																											
Yelp2018		<table><tr><td></td><td>Our results</td><td>Paper</td></tr><tr><td>R@5</td><td>.0277</td><td>.0275</td></tr><tr><td>N@5</td><td>.0534</td><td>.0533</td></tr></table>		Our results	Paper	R@5	.0277	.0275	N@5	.0534	.0533	<table><tr><td></td><td>Our results</td><td>Paper</td></tr><tr><td>R@10</td><td>.0473</td><td>.0474</td></tr><tr><td>N@10</td><td>.0557</td><td>.0558</td></tr></table>		Our results	Paper	R@10	.0473	.0474	N@10	.0557	.0558	<table><tr><td></td><td>Our results</td><td>Paper</td></tr><tr><td>R@20</td><td>.0782</td><td>.0792</td></tr><tr><td>N@20</td><td>.0664</td><td>.0667</td></tr></table>		Our results	Paper	R@20	.0782	.0792	N@20	.0664	.0667
		Our results	Paper																												
	R@5	.0277	.0275																												
	N@5	.0534	.0533																												
	Our results	Paper																													
R@10	.0473	.0474																													
N@10	.0557	.0558																													
	Our results	Paper																													
R@20	.0782	.0792																													
N@20	.0664	.0667																													
Foursquare		<table><tr><td></td><td>Our results</td><td>Paper</td></tr><tr><td>R@5</td><td>.0562</td><td>.0561</td></tr><tr><td>N@5</td><td>.0879</td><td>.0879</td></tr></table>		Our results	Paper	R@5	.0562	.0561	N@5	.0879	.0879	<table><tr><td></td><td>Our results</td><td>Paper</td></tr><tr><td>R@10</td><td>.0859</td><td>.0867</td></tr><tr><td>N@10</td><td>.0901</td><td>.0898</td></tr></table>		Our results	Paper	R@10	.0859	.0867	N@10	.0901	.0898	<table><tr><td></td><td>Our results</td><td>Paper</td></tr><tr><td>R@20</td><td>.1300</td><td>.01314</td></tr><tr><td>N@20</td><td>.1064</td><td>.01068</td></tr></table>		Our results	Paper	R@20	.1300	.01314	N@20	.1064	.01068
		Our results	Paper																												
	R@5	.0562	.0561																												
	N@5	.0879	.0879																												
	Our results	Paper																													
R@10	.0859	.0867																													
N@10	.0901	.0898																													
	Our results	Paper																													
R@20	.1300	.01314																													
N@20	.1064	.01068																													
Douban -book		<table><tr><td></td><td>Our results</td><td>Paper</td></tr><tr><td>R@5</td><td>.1046</td><td>.1055</td></tr><tr><td>N@5</td><td>.2025</td><td>.2018</td></tr></table>		Our results	Paper	R@5	.1046	.1055	N@5	.2025	.2018	<table><tr><td></td><td>Our results</td><td>Paper</td></tr><tr><td>R@10</td><td>.1493</td><td>.1504</td></tr><tr><td>N@10</td><td>.1917</td><td>.1918</td></tr></table>		Our results	Paper	R@10	.1493	.1504	N@10	.1917	.1918	<table><tr><td></td><td>Our results</td><td>Paper</td></tr><tr><td>R@20</td><td>.2075</td><td>.2085</td></tr><tr><td>N@20</td><td>.1959</td><td>.1960</td></tr></table>		Our results	Paper	R@20	.2075	.2085	N@20	.1959	.1960
		Our results	Paper																												
	R@5	.1046	.1055																												
	N@5	.2025	.2018																												
	Our results	Paper																													
R@10	.1493	.1504																													
N@10	.1917	.1918																													
	Our results	Paper																													
R@20	.2075	.2085																													
N@20	.1959	.1960																													
ML-1M		<table><tr><td></td><td>Our results</td><td>Paper</td></tr><tr><td>R@5</td><td>.1225</td><td>.1220</td></tr><tr><td>N@5</td><td>.3499</td><td>.3506</td></tr></table>		Our results	Paper	R@5	.1225	.1220	N@5	.3499	.3506	<table><tr><td></td><td>Our results</td><td>Paper</td></tr><tr><td>R@10</td><td>.1928</td><td>.1933</td></tr><tr><td>N@10</td><td>.3279</td><td>.3281</td></tr></table>		Our results	Paper	R@10	.1928	.1933	N@10	.3279	.3281	<table><tr><td></td><td>Our results</td><td>Paper</td></tr><tr><td>R@20</td><td>.2905</td><td>.2907</td></tr><tr><td>N@20</td><td>.3278</td><td>.3261</td></tr></table>		Our results	Paper	R@20	.2905	.2907	N@20	.3278	.3261
		Our results	Paper																												
	R@5	.1225	.1220																												
	N@5	.3499	.3506																												
	Our results	Paper																													
R@10	.1928	.1933																													
N@10	.3279	.3281																													
	Our results	Paper																													
R@20	.2905	.2907																													
N@20	.3278	.3261																													

Table 1: Results from our implementation vs results from CaDRec’s paper.

Our implementation results fall in line with those as presented in the paper indicating that

the model in the paper is reproducible as well as consistent with a small level of variability. This result provides confidence that the approach in the paper is robust and can be applied across contexts and can be generalized to other datasets, strengthening the reliability of their model.

Further, in their experimentation, they performed four ablation tests which focus on either removing or modifying a specific part of the model to analyze how much of an effect it has on the model's performance. In particular, the ablation tests they performed are without self-attention, disengagement, normalization, and weight sharing. In our experimentation with the ablation test, we only successfully recreated one of the four tests, the one involving removing weight sharing. We believe that the differences in outcome are due to the hardware and environmental setup. In particular, differences in either GPU or CPU could result in differences in training times, convergence, or variation in optimization. Different versions of libraries such as PyTorch, CUDA, NUMPY, etc. could lead to differences in precision, performance, or behavior. Additionally, despite using a set random seed, other small differences in operation could affect the model's performance.

Datasets	R@20, N@20		
Yelp2018		Our results	Paper
	w/oSA	.0779, .0655	.0775, .0649
	w/oDis	.0776, .0658	.0759, .0632
	w/oER	.0727, .0613	.0720, .0605
	w/oWS	.0787, .0665	.0751, .0622
Foursquare		Our results	Paper
	w/oSA	.1302, .1060	.1282, .1039
	w/oDis	.1251, .1025	.1238, .1010
	w/oER	.1198, .0989	.1217, .1002
	w/oWS	.1307, .1063	.1273, .1044
Douban-Book		Our results	Paper
	w/oSA	.2040, .1941	.1954, .1846
	w/oDis	.2007, .1944	.1933, .1816
	w/oER	.1685, .1658	.1713, .1632
	w/oWS	.2081, .1960	.2038, .1960
ML-1M		Our results	Paper

	w/oSA	.2894, .3254	.2907, .3261
	w/oDis	.2700, .3073	.2593, .2913
	w/oER	.2713, .3095	.2580, .2921
	w/oWS	.2882, .3267	.2795, .3152

Table 2: Results from our ablation tests vs results from CaDRec's paper.

To end phase 2, we attempted to migrate CaDRec from using PyTorch to using TensorFlow instead. Our workflow for implementing this migration was to read through the entire source code and whenever a PyTorch specific function was found, we would replace it with TensorFlow's equivalent as well as modify any surrounding code so that it fits with the new format. The primary reasons we considered this reimplementaiton are TensorFlow's native support for TPU use which would increase efficiency for matrix operations of CPUs or GPUs, increased efficiency through conversion into a static graph which could be optimized by its accelerated linear algebra compiler, better ecosystem for both deployment and production as well as its integrated visualization tool for debugging, and its widespread use in industry allow for easier integration into current systems. This reimplementaiton ultimately did not fully work. We managed to modify it such that it was using TensorFlow and would run both training and testing epochs as well as return and record metrics, however the metrics generated by this model were significantly different from CaDRec's paper. Additionally from our experimentation with it, in order for TensorFlow to use a GPU, at least on Palmetto On Demand, a user needs to either be an admin or have elevated privileges of some kind. As such, the runtime of the model was significantly longer than the base model using PyTorch, about 9 minutes to run 1 training epoch where the base PyTorch model took about .776 minutes on its longest dataset.

```

I rendezvous is aborting with status: OUT_OF_RANGE: End of sequence
(Training) P@k:[0.00142, 0.00098, 0.00214, 0.00209], R@k:[0.00022, 0.00081, 0.00502, 0.00924],
(Training)map@k:[0.0, 0.0, 0.0, 0.0], ndcg@k:[0.00142, 0.0012, 0.00309, 0.00446], elapse:8.979 min
([0.00142, 0.00098, 0.00214, 0.00209], [0.00022, 0.00081, 0.00502, 0.00924], [0.0, 0.0, 0.0, 0.0], [0.00142, 0.0012, 0.00309, 0.00446])
(Test) P@k:[0.00202, 0.00189, 0.00442, 0.00429], R@k:[0.00011, 0.00081, 0.00507, 0.00884],
(Test)map@k:[0.0, 0.0, 0.0, 0.0], ndcg@k:[0.00202, 0.00188, 0.00459, 0.00591], elapse:0.655 min
[ Epoch 3 ]
- (Training) : 396it [08:58, 4.04it/s]2025-03-15 20:29:08.879640: I tensorflow/core/framework/local_rendezvous.cc:407] Local rendezvous is aborting with status: OUT_OF_RANGE: End of sequence
(Training) P@k:[0.00142, 0.00098, 0.00214, 0.00209], R@k:[0.00022, 0.00081, 0.00502, 0.00924],
(Training)map@k:[0.0, 0.0, 0.0, 0.0], ndcg@k:[0.00142, 0.0012, 0.00309, 0.00446], elapse:8.984 min
([0.00142, 0.00098, 0.00214, 0.00209], [0.00022, 0.00081, 0.00502, 0.00924], [0.0, 0.0, 0.0, 0.0], [0.00142, 0.0012, 0.00309, 0.00446])
(Test) P@k:[0.00202, 0.00189, 0.00442, 0.00429], R@k:[0.00011, 0.00081, 0.00507, 0.00884],
(Test)map@k:[0.0, 0.0, 0.0, 0.0], ndcg@k:[0.00202, 0.00188, 0.00459, 0.00591], elapse:0.652 min
[ Epoch 4 ]
(Training) P@k:[0.00142, 0.00098, 0.00214, 0.00209], R@k:[0.00022, 0.00081, 0.00502, 0.00924],
(Training)map@k:[0.0, 0.0, 0.0, 0.0], ndcg@k:[0.00142, 0.0012, 0.00309, 0.00446], elapse:8.987 min
([0.00142, 0.00098, 0.00214, 0.00209], [0.00022, 0.00081, 0.00502, 0.00924], [0.0, 0.0, 0.0, 0.0], [0.00142, 0.0012, 0.00309, 0.00446])
(Test) P@k:[0.00202, 0.00189, 0.00442, 0.00429], R@k:[0.00011, 0.00081, 0.00507, 0.00884],
(Test)map@k:[0.0, 0.0, 0.0, 0.0], ndcg@k:[0.00202, 0.00188, 0.00459, 0.00591], elapse:0.654 min
[ Epoch 5 ]
(Training) : 396it [08:57, 4.04it/s]

```

Figure 1: Example run from our TensorFlow version of CaDRec

During phase 3, we were tasked with finding and implementing a non-trivial improvement of the model suggested in the paper. For this task we attempted a variety of methods to improve

the model on either its performance metrics, recall or normalized discounted cumulative gain, its runtime efficiency, or both. All of our results can be found in tables 3-6 below this section.

Our first attempt used adaptive fusion through gating. We attempted this method because CaDRec makes use of two contexts sequential and structural, however it treats them as additive or indirectly combined. In theory this enhancement would allow the model to adaptively balance the influence of the contexts per-instance rather than equally. This idea comes from the idea that some users may be more likely to follow the structural context, item popularity or occurrence patterns, while others may be more likely to follow a sequential context, recent interactions or changing preferences. The implementation of a gating mechanism would allow the model to learn a soft weight between the two contexts and adjust for each input. This was expected to improve the model's ability to personalize results. In our results we find that this adaptive fusion mechanism did not assist any performance metric on any of the datasets. This is likely due to the gating layer adding more complexity without enough regularization or training to learn meaningful gates.

Our next attempt used contrastive learning with hard negative mining. We attempted this method because CaDRec lacks explicit representation learning that encourages discrimination in embeddings. Contrastive learning wants to bring positive pairs closer together and separate negative pairs. Its addition can potentially be beneficial to sparse datasets lacking in explicit labels. Hard negative mining improves upon contrastive learning by forcing the contrastive loss to deal with more informative and difficult examples which will potentially speed up convergence and improve sharpness. We would expect this enhancement to improve the ranking metrics R and NDCG by better separating relevant and non-relevant items. Our testing found that, in terms of performance metrics, the addition of contrastive learning neither helped nor hindered the model, however it did significantly increase the runtime of the model especially so on the Yelp2018 dataset where it nearly doubled to runtime.

Similar to the last attempted enhancement, we next tried contrastive learning with a multi-view head. This follows on from the previous attempt with the difference being that we want to compare representations from different layers of the encoder. This should help to provide supervision, encouraging consistency throughout the network. We expected this enhancement to increase consistency and improve gradient flow and stability. The base model only uses the final layer for prediction and embedding, so implementing this should allow for the exploitation of depth in the structure to generate informative signals. Our testing found that, similarly to contrastive learning with hard negatives, this modification does not significantly affect performance metrics while causing a fairly sharp increase in runtime.

Next we attempted to implement causal debiasing through counterfactual regularization. We attempted this enhancement because recommender systems can become biased due to incorrect correlations from training data leading to poor generalization or underrepresentation or user behaviors. Through this enhancement we want to penalize over-reliance on correlations and encourage stability between a factual and counterfactual context. We expect this enhancement to also improve the fairness of recommendations as well as reduce overfitting to correlations. Our results show that it also does not cause a significant change either positively or negatively, however we do continue to see a fairly large increase in runtime caused by running a second forward pass for the counterfactual context.

Our next attempted enhancement was time-weighted embeddings. We attempted this enhancement because non-historical events are equally important. In general, recent events tend to have more predictive power than older events. CaDRec treats all events in a sequence as equal, instead relying on attention and graph structure to determine importance. We want to implement an explicit temporal decay to reduce how much power older events have in determining a prediction. This enhancement is expected to improve the prediction of next-item interactions by giving more priority to recent behaviors. Our results show that performance metrics drop slightly across most datasets, however we see a larger reduction in runtime suggesting that while the model overall performs worse its time-efficiency may make up for it depending on the end user's needs.

Next, we attempt to enhance the model through implementing sequence trimming. We wanted to try this strategy to see if the model can continue to perform well with less information such as potentially redundant or low-grade events. We expect that this modification will reduce the time cost of training and inference while also removing older or potentially redundant interactions that may reduce performance. We found, through our testing, that performance dropped considerably on both ML-1M and Douban-book while also still dropping though less significantly on Yelp2018 and FourSquare. This is likely due to ML-1M and Douban-book being longer and more dense datasets. Additionally, we did see a runtime reduction, but not by a significant enough margin to make up for the performance loss.

Next, we attempted to implement session-level attention. We attempted this method because, CaDRec currently treats all positions as equally important when aggregating representations however not all events are equally important in a session so, like other modifications, we want to give more weight to recent or more relevant items. We expect that this modification will better tailor user representations to recent actions as well as show better performance over short sequences. In our testing we found that Yelp2018 and FourSquare had a fair drop in performance while having a decently larger improvement in runtime making it a potentially acceptable modification for those datasets. On the other hand, Douban-book and ML-1M saw a massive reduction in performance having near-zero R and NDCG. This is likely due to under-training, softmax collapse or padding mismatches causing instability.

Next, we attempted to implement adjacency matrix sampling. We implemented this because CaDRec makes use of dense per-user subgraphs by extracting a fully connected sub-adjacency matrix from a global user-item graph which can introduce weak or redundant connections and be computationally expensive. We expect that this modification will reduce computation overhead, improve generalization, retain neighbors, and improve scalability. In implementation, we found that this change reduced performance on all datasets, especially so for ML-1M suggesting that its model strongly benefits from full structural context. Additionally, it suggests that naive top-k filtering may remove valuable structure, especially from large-item domains. In regards to runtime, we see that this model in fact runs faster across all datasets. The particular drops in performance and runtime suggest that this implementation could be useful for the FourSquare dataset as it sees a relatively small drop in performance but a fairly large improvement in runtime.

Lastly, we attempted to implement a knowledge distillation strategy through use of separate teacher and student models. We implemented this strategy due to computational expense of deep recommender systems in real-time systems. Instead, a full model's

representations can be transferred to a smaller model. This would allow for a full model, the teacher, to train a smaller model, the student, to mimic it using logits from the teacher as target. We expect this modification to reduce training and inference time while preserving the performance of the teacher model. In our results we found that performance metrics dropped across all datasets, especially so for Yelp2018 and Douban-book. At the same time, training time and testing time also drop modestly for all datasets. This indicates that the knowledge distillation strategy would likely be useful for datasets similar to FourSquare or ML-1M while, depending on importance of performance metrics, still being useful on other datasets to be more computationally efficient.

Yelp2018	Base Model	Adaptive Fusion through Gating	Contrastive Learning Augmentation + Hard Negative Mining	Contrastive Learning + Multi-View Head	Counterfactual Regularization	Time-Weighted Embeddings	Sequence Trimming	Session-Level Attention	Adjacency Matrix Sampling	Knowledge Distillation / Teacher - Student
Runtime per Epoch	Training $\approx$.776 minutes Testing $\approx$.395 minutes	Training $\approx$.700 minutes Testing $\approx$.351 minutes	Training $\approx$ 1.550 minutes Testing $\approx$.718 minutes	Training $\approx$ 1.327 min Testing $\approx$ 0.624 min	Training $\approx$ 1.107 min Testing $\approx$ 0.667 min	Training $\approx$.661 minutes Testing $\approx$.333 minutes	Training $\approx$.649 minutes Testing $\approx$.323 minutes	Training $\approx$.679 minutes Testing $\approx$.344 minutes	Training $\approx$.688 minutes Testing $\approx$.356 minutes	First Half Training $\approx$.716 minutes Testing $\approx$.340 minutes Second Half Training $\approx$.642 minutes Testing $\approx$.338 minutes
R@5	.0275	.02559	.0277	.0272	.0277	.02718	.02562	.02629	.02511	.02434
NDCG@5	.0533	.0493	.0535	.0530	.0632	.05258	.04687	.05048	.04926	.04745
R@10	.0474	.04399	.0471	.0470	.0468	.04647	.04456	.04452	.04347	.04223
NDCG@10	.0558	.05169	.0557	.0556	.0535	.05475	.04995	.05258	.05149	.04983.
R@20	.0792	.07407	.0793	.0790	.0785	.07761	.07491	.07515	.07342	.07112
NDCG@20	.0667	.0623	.0669	.0670	.0665	.06564	.06133	.0634	.06184	.05996

FourSquare	Base Model	Adaptive Fusion through Gating	Contrastive Learning Augmentation + Hard Negative Mining	Contrastive Learning + Multi-View Head	Counterfactual Regularization	Time-Weighted Embeddings	Sequence Trimming	Session-Level Attention	Adjacency Matrix Sampling	Knowledge Distillation / Teacher - Student
Runtime per Epoch	Training $\approx$.185 minutes Testing $\approx$	Training $\approx$.232 minutes Testing $\approx$	Training $\approx$.292 minutes Testing $\approx$	Training $\approx$.314 min Testing $\approx$	Training $\approx$.254 min Testing $\approx$	Training $\approx$.158 Testing $\approx$.077	Training $\approx$.160 Testing $\approx$.079	Training $\approx$.163 minutes Testing $\approx$	Training $\approx$.166 minutes Testing $\approx$	First Half Training $\approx$.170 minutes

	.091 minutes	.125 minutes	.130 minutes	.145 min	.146 min			.081 minutes	.084 minutes	Testing $\approx$.080 Second Half Training $\approx$.152 minutes Testing $\approx$.080
R@5	.0561	.0558	.0552	.0560	.0558	.0556	.05426	.05386	.0541	.05268
NDCG@5	.0879	.0873	.0872	.0877	.087	.0867	.08248	.08419	.08505	.08329
R@10	.0867	.0858	.0857	.0858	.0863	.0853	.08447	.08328	.08521	.08117
NDCG@10	.0898	.0897	.0897	.0896	.0898	.0890	.08615	.8687	.08825	.08546
R@20	.1314	.1295	.1297	.1291	.1299	.1297	.12736	.12746	.12916	.1227
NDCG@20	.1068	.1061	.1062	.1058	.1061	.1057	.1029	.10331	.10476	.10099

Douban-book	Base Model	Adaptive Fusion through Gating	Contrastive Learning Augmentation + Hard Negative Mining	Contrastive Learning + Multi-View Head	Counterfactual Regularization	Time-Weighted Embeddings	Sequence Trimming	Session-Level Attention	Adjacency Matrix Sampling	Knowledge Distillation / Teacher-Student
Runtime per Epoch	Training $\approx$.282 minutes Testing $\approx$.155 minutes	Training $\approx$.773 minutes Testing $\approx$.178 minutes	Training $\approx$.526 minutes Testing $\approx$.228 minutes	Training $\approx$.704 min Testing $\approx$.328 min	Training $\approx$.413 min Testing $\approx$.257 min	Training $\approx$.248 Testing $\approx$.136	Training $\approx$.240 minutes Testing $\approx$.131 minutes	Training $\approx$.249 minutes Testing $\approx$.135 minutes	Training $\approx$.257 minutes Testing $\approx$.145 minutes	First Half Training $\approx$.289 minutes Testing $\approx$.136 Second Half Training $\approx$.259 minutes Testing $\approx$.135
R@5	.1055	.0795	.1045	.1048	.1048	.10007	.09378	.00081	.09914	.08949
NDCG@5	.2018	.1651	.2020	.2024	.2020	.19273	.14835	.00177	.18538	.17474
R@10	.1504	.1195	.1497	.1498	.1491	.14471	.1349	.00507	.14215	.12906
NDCG@10	.1918	.1565	.1915	.1924	.1911	.18385	.14709	.00667	.17791	.16622
R@20	.2085	.1738	.2082	.2073	.2065	.20122	.18555	.00884	.19856	.17944
NDCG@20	.1960	.1614	.1960	.1963	.1950	.18846	.15871	.00658	.18359	.16986

ML-1M	Base Model	Adaptive Fusion through Gating	Contrastive Learning Augment	Contrastive Learning +	Counterfactual Regularization	Time-Weighted Embeddings	Sequence Trimming	Session-Level Attention	Adjacency Matrix Sampling	Knowledge Distillation /
-------	------------	--------------------------------	------------------------------	------------------------	-------------------------------	--------------------------	-------------------	-------------------------	---------------------------	--------------------------

			ation + Hard Negative Mining	Multi-Vie w Head						Teacher - Student
Runtime per Epoch	Training ≈ .129 minutes Testing ≈ .079 minutes	Training ≈ .197 minutes Testing ≈ .105 minutes	Training ≈ .310 minutes Testing ≈ .128 minutes	Training ≈ .332 minutes Testing ≈ .147 minutes	Training ≈ .229 min Testing ≈ .144 min	Training ≈ .117 minutes Testing ≈ .070 minutes	Training ≈ .108 minutes Testing ≈ .063 minutes	Training ≈ .117 minutes Testing ≈ .070 minutes	Training ≈ .120 minutes Testing ≈ .073 minutes	First Half Training ≈ .138 minutes Testing ≈ .066 Second Half Training ≈ .123 minutes Testing ≈ .066
R@5	.1220	.1153	.1224	.1218	.1230	.1202	.09526	.00905	.09276	.11847
NDCG@5	.3506	.3394	.3499	.3514	.3508	.3488	.24677	.03099	.28509	.34299
R@10	.1933	.1861	.1934	.1912	.1939	.1918	.14558	.01779	.14062	.18665
NDCG@1 0	.3281	.3179	.3279	.3269	.3285	.3270	.22621	.03099	.25196	.32099
R@20	.2907	.2787	.2912	.2867	.2885	.2853	.17935	.03544	.16915	.27985
NDCG@2 0	.3261	.3156	.3280	.3257	.3270	.3243	.20668	.03077	.21539	.31923

Table 3-6: Performance and Runtime Results from all attempted enhancements.

From all of our work in researching, implementing, and experimenting with the CaDRec model we can confidently say that it is a well made model that is on the edge of recommender system research. On its own it makes use of novel methods to improve already existing recommender systems and as such discovering a method or enhancement that will directly improve either its performance or efficiency will be a large innovation. While it is not truly an innovation, one additional change we made was reducing the number of times that the model is evaluated from every epoch to once every fifth epoch or if the last epoch is not divisible by five, the last epoch as well. This necessarily improved runtime efficiency by reducing total testing time to one fifth of its original time, for example in training the base model on ML-1M, you would spend about 2.37 minutes testing the model, .079 minutes per epoch over 30 epochs. By only testing every fifth you reduce testing time to .474 minutes to complete 30 epochs. This comes at the cost of potentially missing the best model embeddings, however the embeddings that are found are likely to still be within an acceptable range. Given more time to continue working, we would like to spend more time testing these current enhancements with each other as well as spend more time tuning their hyperparameters and the overall hyperparameters of the model to determine if there were ways to improve their performance above that of the base model.